Rapport

Disa Degerholm, Hampus Hillding, Anton Fornstedt, Fredrik Sollenberg

29 oktober 2026

Uppgift 3. Existens men inte entydighet

Uppgift 3. Disa Degerholm

Betrakta begynnelsevärdesproblemet

$$y' = \sqrt{|y|}, \quad y(0) = 0. \tag{1}$$

1. Analytisk lösning och icke-entydighet

Picards existens- och entydighetssats (Simmons kap. 13) kräver att $f(y) = \sqrt{|y|}$ är lokalt Lipschitzkontinuerlig i en omgivning av $y = 0$, dvs. att differenskvoten är begränsad:

$$\lim_{y \to 0^+} \frac{|f(y) - f(0)|}{|y - 0|} = \lim_{y \to 0^+} \frac{\sqrt{y}}{y} = \lim_{y \to 0^+} \frac{1}{\sqrt{y}} = \infty. \tag{2}$$

Eftersom Lipschitzvillkoret fallerar i $y = 0$ upphör entydigheten (Peanos existenssats garanterar endast existens).

Separation av variabler för $y \neq 0$:

- För $y > 0$: $\int y^{-1/2}\, dy = \int dt \implies 2\sqrt{y} = t - c_2 \implies y(t) = \frac{1}{4}(t - c_2)^2$ $(t \geq c_2)$.
- För $y < 0$: $\int (-y)^{-1/2}\, dy = \int dt \implies -2\sqrt{-y} = t - c_1 \implies y(t) = -\frac{1}{4}(t - c_1)^2$ $(t \leq c_1)$.

Eftersom $y' \geq 0$ är $y(t)$ icke-avtagande och kan ligga kvar på jämvikten $y \equiv 0$ under ett godtyckligt tidsintervall $[c_1, c_2]$:

$$y(t) = \begin{cases} -\frac{1}{4}(t - c_1)^2, & t \leq c_1, \\ 0, & c_1 < t < c_2, \\ \frac{1}{4}(t - c_2)^2, & t \geq c_2, \end{cases} \quad \text{där } c_1 \leq 0 \leq c_2. \tag{3}$$

Detta ger en tvåparametrig familj av oändligt många lösningar.

2. Numerisk lösning med RK4

Med $y_0 = 0$ fås i första RK4-steget:

$$k_1 = f(0) = 0 \implies k_2 = k_3 = k_4 = 0 \implies y_1 = y_0 = 0. \tag{4}$$

Genom induktion gäller $y_n = 0$ för alla n. En standardlösare förblir deterministiskt låst i den stationära lösningen $y(t) \equiv 0$ och kan inte numeriskt detektera förgreningen utan yttre störning.

3. Hur länge lösningen fastnar vid $y(-1) = -1$

För $y < 0$ ger separation av variabler med $y(-1) = -1$:

$$-2\sqrt{-y} = t + C \implies -2\sqrt{1} = -1 + C \implies C = -1, \tag{5}$$

vilket ger $y(t) = -\frac{1}{4}(1 - t)^2$ för $t \leq 1$. Analytiskt nås $y = 0$ vid $t = 1$, varpå lösningen kan kvarstanna i $y \equiv 0$ godtyckligt länge.

- **Tid till tröskeln $|y| \le 10^{-6}$:** För $y < 0$ ger separation av variabler med $y(-1) = -1$:

$$\int (-y)^{-1/2}\, dy = \int dt \implies -2\sqrt{-y} = t - 1 \implies y(t) = -\frac{1}{4}(1-t)^2.$$

  Tröskelvillkoret $|y| \le 10^{-6}$ ger:

$$\frac{1}{4}(1-t)^2 \le 10^{-6} \implies |1-t| \le 2 \cdot 10^{-3} \implies t \approx 0{,}998.$$

  Tiden från startpunkten $t = -1$ blir därmed $\Delta t = 0{,}998 - (-1) = 1{,}998$.

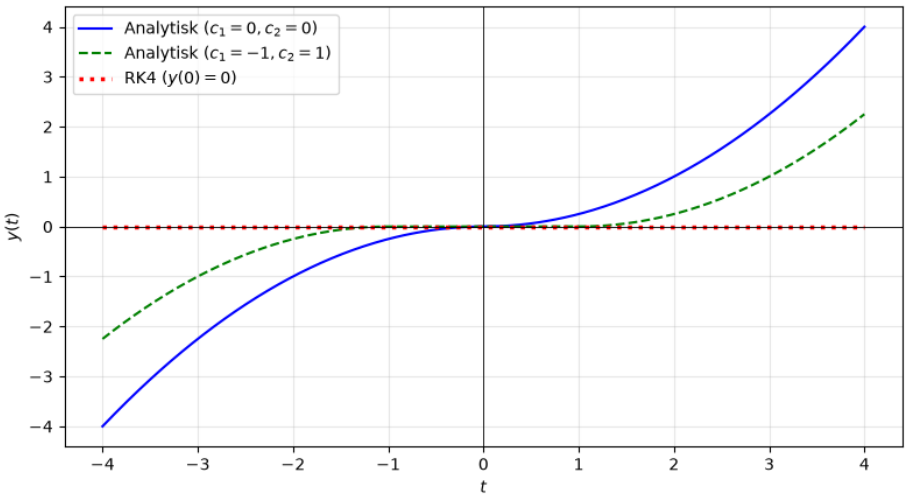
- **Numeriskt beteende:**
  - Om $y_n = 0$ exakt blir alla lutningar noll:

$$k_1 = f(t_n, 0) = \sqrt{|0|} = 0 \implies k_2 = k_3 = k_4 = 0 \implies y_{n+1} = y_n = 0,$$

    vilket låser lösaren permanent i jämvikten.
  - Om avrundningsfel ger $y_n = \delta > 0$ fås efter ett Euler-steg (eller motsvarande för RK4):

$$y_{n+1} = \delta + h\sqrt{\delta} \implies \frac{y_{n+1}}{y_n} = 1 + \frac{h}{\sqrt{\delta}}.$$

    Eftersom $\lim_{\delta \to 0^+} \frac{h}{\sqrt{\delta}} = \infty$ växer tillskottet $\Delta y = h\sqrt{\delta} \gg \delta$ explosionsartat, varför lösningen lämnar intervallet $|y| \le 10^{-6}$ på ett fåtal steg.



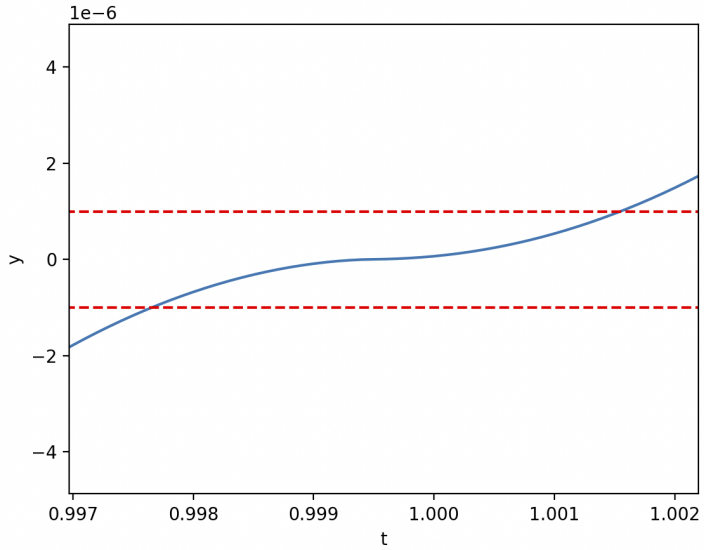

```
eps = 0.0e+00: fastnar i 0.0039 s
eps = 1.0e-12: fastnar i 0.0039 s
eps = 1.0e-08: fastnar i 0.0040 s
eps = 1.0e-04: fastnar i 0.0040 s
```

Uppgift 3. Anton Fornstedt

Analytisk lösning. Förutom $y \equiv 0$ löser $y(t) = 0$ för $0 \le t \le c$ och $y(t) = (t-c)^2/4$ för $t > c$ problemet för varje $c \ge 0$, så det finns oändligt många lösningar. Picard–Lindelöf garanterar ingen entydighet eftersom $f(y) = \sqrt{|y|}$ inte är Lipschitz nära 0: $|f(y) - f(0)|/|y| = 1/\sqrt{|y|} \to \infty$.

Numerisk lösning. Euler ger $y_1 = 0 + h\sqrt{0} = 0$, och likaså alla följande steg, så den numeriska lösningen blir $y \equiv 0$ oavsett h.

Hur länge lösningen fastnar. Med $y(-1) = -1 + \varepsilon$ och $h = 10^{-6}$ låg lösningen i $|y| \le 10^{-6}$ under $\Delta t \approx 0{,}003998$, både för $\varepsilon = 0$ och $\varepsilon = 0{,}1$. Analytiskt gäller $(t-t_0)^2/4 \le 10^{-6} \iff |t-t_0| \le 2\cdot 10^{-3}$, alltså $\Delta t = 4\cdot 10^{-3}$, så lösningen fastnar inte. Eftersom ekvationen är autonom flyttar ε bara nollgenomgången t_0, inte tiden nära noll.



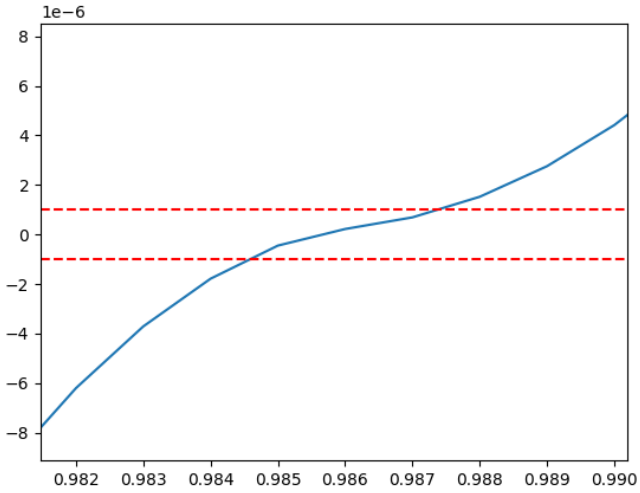
Figur 1: Numerisk lösning nära $y = 0$. Streckade linjer: $|y| = 10^{-6}$.

Uppgift 3. Hampus Hillding

Analytisk lösning: Oändligt många lösningar finns där $y(t)$ har formen:

- $y(t) = \frac{(t-a)^2}{4}$ då $t < a$
- $y(t) = 0$ då $t \in [a, b]$
- $y(t) = -\frac{(t-b)^2}{4}$ då $t > b$

För konstanter $a, b : a < 0 < b$.
Numeriskt: Min numeriska lösning stannar på $y = 0$ för alltid.
Med begynnelsevillkor $y(-1) = -0.99$ får jag att y ”fastnar” på 0 i ungefär 0.03 enheter av t. Se figur.

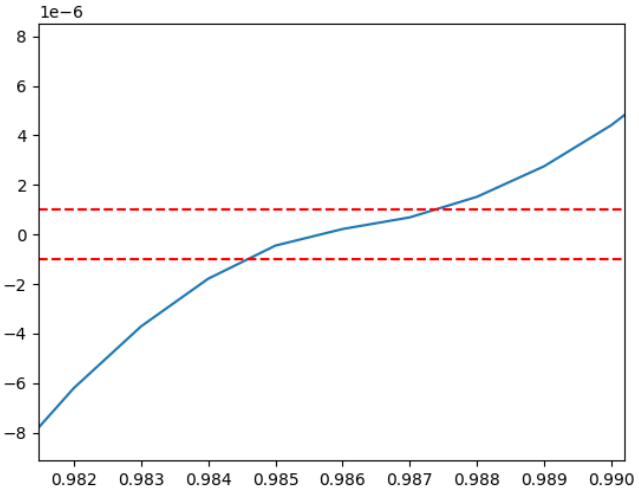


Analytisk lösning. Oändligt många lösningar finns, där $y(t)$ kan ha formen:

- $y(t) = -\frac{(t-a)^2}{4}$ då $t < a$,
- $y(t) = 0$ då $t \in [a, b]$,
- $y(t) = \frac{(t-b)^2}{4}$ då $t > b$,

för konstanter $a < 0 < b$.

Numeriskt. Min numeriska lösning stannar nära $y = 0$. Med begynnelsevillkoret $y(-1) = -0{,}99$ ligger den nära noll i ungefär 0,03 tidsenheter. Se figur.



Uppgift 3. Fredrik Sollenberg

Analytiskt: Betrakta begynnelsevärdesproblemet $\frac{dy}{dt} = \sqrt{|y|}, \quad y(0) = 0$. Vi kan enkelt övertyga oss om att $f(t, y) = \sqrt{|y|}$ är kontinuerlig överallt men eftersom $\frac{\partial f}{\partial y} \to \infty$ (inte Lipschitz) vid $t = 0$ kan lösningars existens garanteras men de behöver inte vara unika vid $y = 0$.

Separering och integrering av D.E:n ger lösningen $y = \pm(\frac{t}{2} + c)^2$, (en parabel som tangerar horisontella axeln), och eftersom $\frac{dy}{dt} = \sqrt{|y|} \geq 0$ för alla t har vi bara den "uppåtlutande" delen av kurvan. Vi kan föreställa oss resultatet som en $y = x^3$-liknande kurva som börjar i det negativa, stiger till en terasspunkt

i $y = 0$ och fortsätter uppåt. Men... om y kommer till 0 så har vi $y'(0) = \sqrt{|0|} = 0$ och den förebådade tvetydigheten uppenbarar sig.

Numeriskt: I vårt fall när $y(0) = 0$ kommer en numerisk lösningsmetod för begynnelsevärdesproblem att spotta tillbaka en kurva där y alltid är lika med 0. Exempelvis, för framåt-euler-metoden kommer lösaren att utföra beräkningen

$$y_{n+1} = 0 + \Delta t\sqrt{0} = 0$$

som aldrig kommer att lämna $y(0) = 0$. Detsamma går att visa för RK4. Vi vet att om $y(0) > 0$ går kurvan bestämt uppåt mot ∞. Däremot om $y(0) < 0$ kommer den att gå uppåt mot 0. Vi kan bevisa att kurvan numeriskt kommer att slå över till den positiva sidan (givet att inget steg landar på exakt 0). Låt oss säga $|y_n| = \delta > 0$. Villkoret för övergången är

$$y_{n+1} = -\delta + \Delta t\sqrt{\delta} > 0 \implies \Delta t > \sqrt{\delta}$$

Vilket till slut måste hända för att kurvan, medan den är negativ, alltid kommer att röra sig närmre 0. När ett steg väl är positivt gäller även egenskapen

$$\frac{y_{n+1}}{y_n} = \frac{\delta + \Delta t\sqrt{\delta}}{\delta} = 1 + \frac{\Delta t}{\sqrt{\delta}} \to \infty, \text{ när } \delta \to 0^+$$

vilket innebär att den numeriska lösningskurvan verkligen inte vill fastna kring 0. Detsamma går att visa för metoder som RK4 m.fl.

Observationer m. Python: För det ursprungliga problemet där $y(0) = 0$ får vi likt förväntat en rakt streck eftersom y alltid är lika med 0. För den andra situationen $y(-1) = -1$ får vi en s-kurva som vi förväntade oss. Jag testade med flera metoder och ingen fastnade vid $y = 0$. Mina kamrater beräknade att tiden den analytiska kurvan befinner sig inom $|y| \leq 10^{-6}$ borde vara 0.004 vilket stämde överens med mina test för fram/bak-Euler. Högre ordningens explicita funktioner hade faktiskt en kortare visit. Ingen metod jag provade hade längre.

Uppgift 3. Skillnader och likheter

1. **Skillnader i analytisk lösning av ODE:n:** Disa och Fredrik tittade på Lipschitz kontinuitet i punkten y=0 medan Hampus löste ODE:n direkt. Detta gjorde att när Hampus tittade på sin numeriska lösning antog funktionen 0 inte bara i ett värde utan flera medan Disa hittade det. Anton löste ODE:n direkt som Hampus, men använde även Lipschitz-argumentet som Disa. Han fick en lösningsfamilj med en parameter $c \geq 0$ (bara $t \geq 0$), medan Disa även lät lösningen komma in från $t < 0$ och fick två parametrar.

2. **Lös problemet numeriskt med kommentarer:** Alla i gruppen fick att $y = 0, \forall x \in \mathbb{R}$ oavsett om man använde RK4 eller Euler. Detta kunde förutses: Eftersom $f(0) = 0$ blir varje explicit stegmetod låst i $y = 0$ om den startar exakt där.

3. **Hur länge fastnar den numeriska lösningen:** Disa härledde analytiskt via separation av variabler med startvärdet $y(-1) = -1$ att lösningen ges av $y(t) = -\frac{1}{4}(1-t)^2$ för $t \leq 1$. Analytiskt nås därmed $y = 0$ vid $t = 1$, varpå lösningen kan kvarstanna i jämvikten $y \equiv 0$ godtyckligt länge eftersom Picards entydighetssats upphör att gälla i $y = 0$. Hon noterade att om en numerisk lösare träffar $y_n = 0$ exakt så blir alla steglutningar noll ($k_i = 0$), vilket låser den i jämvikten. Men som Hampus visade växer den numeriska lösningen ändå vidare i praktiken, vilket förklaras av att flyttalsavrundningar ger ett litet fel $y_n = \delta > 0$ där den relativa tillväxten $y_{n+1}/y_n \approx 1 + h/\sqrt{\delta}$ exploderar då $\delta \to 0^+$ och skjuter lösningen förbi origo. Anton mätte i stället tiden som lösningen befinner sig i intervallet $|y| \leq 10^{-6}$ och fick $\Delta t \approx 0{,}004$, vilket stämmer med det analytiska värdet $4 \cdot 10^{-3}$ och är oberoende av ε.

Uppgift 4. PANG!

Uppgift 4. Disa Degerholm

Betrakta begynnelsevärdesproblemet

$$y' = y^2, \quad y(0) = y_0 = \frac{N}{100}. \tag{6}$$

1. Analytisk lösning och singularitet

För $y \neq 0$ ger separation av variabler med $y(0) = y_0$:

$$\int y^{-2}\,dy = \int dt \implies -\frac{1}{y} = t - \frac{1}{y_0} \implies y(t) = \frac{y_0}{1 - y_0 t} = \frac{N}{100 - Nt}. \tag{7}$$

Lösningen har en vertikal asymptot (smäller) då nämnaren försvinner:

$$t^* = \frac{1}{y_0} = \frac{100}{N}. \tag{8}$$

För $N = 82$ är den exakta singularitetstiden $t^* = \frac{100}{82} \approx 1{,}219512$.

2. Numerisk singularitet och skattning av t^*

- **Numeriskt bevis för singularitet:** Vid numerisk integration (t.ex. RK4) växer y_n och derivatan $y_n' = y_n^2$ explosionsartat då $t \to t^*$. Eftersom lösningskurvan blir vertikal kan ett fixt steg h inte följa funktionen. Värdena överskrider snabbt gränsen för standardflyttal (`float64` $\sim 1{,}8 \cdot 10^{308}$) och genererar `inf`/`NaN`, vilket numeriskt påvisar singulariteten.

- **Uppskattning av t^*:** Genom att integrera framåt med steglängd $h = 0{,}01$ fram till ett tröskelvärde (t.ex. $y > 10^5$) avbryts beräkningen vid sista steget:

  $$t_{\text{stopp}} \approx 1{,}2100.$$

  Detta ger en numerisk undre uppskattning av singularitetstiden som stämmer överens med det analytiska värdet $t^* = \frac{100}{82} \approx 1{,}2195$. Den numeriska beräkningen ger dock $t_{stopp} \approx 1,23$ vilket ligger nära den analytiska singularitetstiden $t^* \approx 1,2195$. Skillnaden beror på den numeriska metodens steglängd och det valda tröskelvärdet och med mindre h kommer man närmare och närmare 1,2195.

3. Bevis: $y(0) < 0 \implies y(t) < 0$ för alla $t > 0$

Högerledet $f(y) = y^2$ har kontinuerlig partiell derivata $\frac{\partial f}{\partial y} = 2y$, vilket gör f lokalt Lipschitzkontinuerlig. Enligt Picards entydighetssats (Simmons kap. 13) kan två skilda lösningskurvor aldrig skära varandra:

- Funktionen $y(t) \equiv 0$ är en stationär lösning ($0' = 0^2$).
- Om det funnes ett $t_1 > 0$ med $y(t_1) \geq 0$, måste $y(t)$ enligt satsen om mellanliggande värden anta $y(t_0) = 0$ för något $t_0 \in (0, t_1]$.
- I punkten $(t_0, 0)$ skulle banan skära $y \equiv 0$, vilket motsäger Picards entydighet. Alltså gäller $y(t) < 0$ för alla $t > 0$.

4. Numeriskt felsteg och automatlösare

Betrakta $y' = Cy^2$ med $y_0 = \epsilon < 0$. Ett Euler-steg med steglängd h ger:

$$y_1 = \epsilon + hC\epsilon^2 = \epsilon(1 + hC\epsilon). \tag{9}$$

Eftersom $\epsilon < 0$ blir $y_1 > 0$ om faktorn växlar tecken:

$$1 + hC\epsilon < 0 \implies h > \frac{1}{C|\epsilon|}. \tag{10}$$

För t.ex. $C = 50$, $\epsilon = -0{,}10$ och $h = 0{,}5$ ger RK4 ett falskt omslag till $y_1 = 0{,}0704 > 0$, varpå lösningen exploderar mot $+\infty$ istället för att avta mot noll.

Automatlösarens skydd:

- **Adaptiv steglängd:** Felet uppskattas via inbäddade metoder (t.ex. Runge–Kutta–Fehlberg) och h skalas ned så att $h \ll \frac{1}{C|y|}$.
- **Invarians- och teckenkontroll:** Steget förkastas omedelbart om y_n byter tecken ($y_n \cdot y_{n+1} \leq 0$) eftersom teckenbyte är analytiskt förbjudet.

```
Del 1: Exakt t* = 1.2195, Numeriskt t_stopp = 1.2300, Sista y = 3.72e+14
Del 2: y(0) = -0.10 -> Efter 1 steg (h=0.5): y = 0.0704 (felaktigt omslag till > 0)
```

Uppgift 4. Anton Fornstedt

Analytisk lösning. Med $y(0) = N/100 = 0{,}34$ ger separation av variabler $y(t) = 1/(50/17 - t)$, som smäller vid $t^* = 50/17 \approx 2{,}94118$.

Numeriskt. Euler med $h = 10^{-4}$ (avbrott då $y > 10^{10}$) smäller vid $t \approx 2{,}94260$, alltså något senare än analytiskt. Det beror på att lösningen är konvex, så tangenten (Eulersteget) hamnar under kurvan.

$y(0) < 0 \Rightarrow y(t) < 0$**.** Antag att $y(t_2) \geq 0$ för något $t_2 > 0$. Enligt satsen om mellanliggande värden finns då t_1 med $y(t_1) = 0$. Men $y \equiv 0$ är också en lösning genom $(t_1, 0)$, och eftersom y^2 är lokalt Lipschitz ger Picard–Lindelöf att lösningen är entydig. Alltså är $y \equiv 0$, vilket motsäger $y(0) < 0$.

Numeriskt teckenbyte. För $y' = Cy^2$ ger Euler $y_{n+1} = y_n(1 + hCy_n)$. Med $y_0 = \varepsilon < 0$ byter lösningen tecken redan i första steget om

$$C > \frac{1}{h|\varepsilon|},$$

till exempel $C > 10^{14}$ för $\varepsilon = -10^{-10}$ och $h = 10^{-4}$. Därefter exploderar den numeriska lösningen, trots att den exakta lösningen aldrig blir positiv. En automatlösare kan undvika detta genom att förkasta steget och halvera h när y byter tecken.

Uppgift 4. Hampus Hillding

Analytiskt: $N = 98$ ger $y = \frac{-1}{x - \frac{100}{98}}$ som smäller när $x = \frac{100}{98} \approx 1.02$.
Numeriskt: Med hjälp av Eulers metod med steglängd $h = 0.001$ får jag att lösningen efter en stund börjar växa väldigt snabbt och det sista värdet jag får innan jag när 'OverflowError' är $x = 1.036$, $y \approx 2.3 \cdot 10^{210}$. Vilket är lite senare än den riktiga singulariteten. **Bevis:** Eftersom $f(x, y)$ och $\frac{\partial f}{\partial y}$ är kontinuerliga på varje sluten rektangel så gäller Picards sats som säger att det finns en *unik* integralkurva genom en given punkt. Detta innebär att två integralkurvor aldrig kan dela en gemensam punkt (korsa varandra). Eftersom $y(x) = 0$ är en integralkurva kan inga andra integralkurvor gå från y<0 till y>0 (satsen om mellanliggande värden) v.s.b.
Teckenbyte: Med $C = 100000$, $\epsilon = -0.001$ och $h = 0.02$ fick jag y att gå från negativt till positivt.
Automatlösare: För att få en automatlösare att undvika dessa typer av fel skulle man kunna känna av när y är på väg att byta tecken och anpassa steglängden därefter.

Uppgift 4. Fredrik Sollenberg

Analytiskt: Vi har begynnelsevärdesproblemet $\frac{dy}{dt} = y^2, y(0) = N/100$ där N är de två sista siffrorna i ens personnummer. Vi vet att $f(t, y) = y^2$ är kontinuerlig överallt och vi kan dessutom konstatera att funktionen är lokalt Lipschitz överallt. Picard ger: Det finns en unik lösning överallt. Vi löser D.E:n enkelt med separering och integrering och får att $y = \frac{-1}{t+c}$. För vårt positiva startvärde fås lösningskurvan av den positiva delen av lösningen till D.E:n. Vi vet att en kurva är strikt positiv eller strikt negativ (utom $y = 0$) eftersom gränsfallskurvan $y = 0$ fungerar som en mur som vi vet inte kan korsas utan att bryta mot lösningarnas redan fastställda entydighet.

Kurvan kommer att gå mot oändligheten när $t \to c$. Vi kan lösa ut c ur D.E:ns lösningsfunktion och inse att vi analytiskt kan beräkna tiden för singulariteten (t^*) utifrån en enda punkt.

Numeriskt: Vi kan påvisa singulariteten numeriskt genom att försöka lösa begynnelsevärdesproblemet som om ingenting och se hur programmet kraschar eller visar en plot där kurvan är låg men på bara

något enstaka tidssteg går till astronomiska siffror utan att någonsin komma förbi en viss tidpunkt. Om man försöker bestämma t^* genom det sista steget i Eulers metod kommer man att få ett för högt tal eftersom metoden kommer att hamna under den analytiska kurvan och kommer därmed "tro" att t^* kommer senare. Att lösa för c i den analytiska lösnigen för en punkt ger också ett högt värde av samma anledning.

Observationer m. Python: I det här fallet finner vi precis det vi förväntat oss hittils. Om vi nu har $\frac{dy}{dt} = Cy^2, y(0) = \epsilon$ kan för för vissa C och små $\epsilon < 0$ en negativ lösning felaktigt ge en positiv kurva med numeriska metoder. För Eulers metod visar det sig att

$$y_1 = \epsilon + \Delta t C \epsilon^2 = \epsilon(1 + \Delta t C \epsilon),$$

$$\text{Felaktigt positiv om: } 1 + \Delta t C \epsilon < 0 \Leftrightarrow \Delta t |\epsilon| > \frac{1}{C}$$

Alltså, om tidssteget eller C är för stora kan vi få den här effekten.

Uppgift 4. Skillnader och likheter

1. **Lös analytiskt mha personnummer:** Disa löste ODE:n för generellt N och stoppade sedan in sina sista två personnummersiffror. Hampus löste med sina direkt i formeln men båda fick samma svar (om N är samma). Anton gjorde som Hampus. Fredrik gjorde som Hampus som gjorde som Anton.

2. **När smäller det numeriskt?** Hampus, Fredrik och Anton använde Eulers metod medan Disa använde RK4. För begynnelsevärdesproblemet $y' = y^2$ är lösningen $y(t) = \frac{y_0}{1 - y_0 t}$ strikt konvex ($y'' = 2yy' = 2y^3 > 0$). Eulers metod approximerar kurvan med tangenten $y_{n+1} = y_n + hy_n^2$, vilken på grund av konvexiteten ligger strikt under den sanna lösningskurvan. Euler underskattar därför tillväxten kraftigt i varje steg och släpar efter, vilket gör att Euler integrerar förbi den teoretiska singulariteten ($t^* = 1/y_0$) och kraschar först efter den exakta tiden (som Anton observerade med $t \approx 2{,}94 > t^*$). Fredrik utnyttjade också kännedomen om den analytiska lösningen och hittade singulariteten utifrån en punkt på lösningskurvan för att få en ytterligare gissnig för t^*.

   RK4 tar däremot delsteg längre fram i intervallet där lutningen $f(t, y) = y^2$ redan har hunnit växa kraftigt. På grund av den höga ordningen $\mathcal{O}(h^4)$ följer RK4 den explosiva tillväxten mycket mer precist och nådde tröskeln $y \geq 10^5$ strax före den analytiska singulariteten $t^* \approx 1{,}1$ innan flyttalsöverflöde (`overflow`) inträffade. Att Disa kunde ta fler eller färre steg beror helt på vald steglängd h och avbrottströskel. Den numeriska smälltiden beror alltså på metodens ordning och valda stoppvillkor, och man kan inte lita på numeriska lösare i omedelbar närhet av en vertikal asymptot.

3. **Visa att om $y(0) < 0$ så måste $y(t) < 0$ för alla $t > 0$:** Alla i gruppen hade samma argument med att f är lokalt Lipschitz kontineruligt överallt och att då y(t)=0 är en lösning kan ingen annan lösning korsa den och blir därmed bara över eller under 0 i sitt y värde.

4. **Numeriska exempel:** Hampus testade sig fram till ett stort nog C då han tänkte att ett stort C skulle ge ett y' som gav tillräckligt stort hopp för att komma över gränsen. Disa härledde formeln för det istället och kom fram till att $h > \frac{1}{C|\epsilon|}$. Anton härledde samma villkor som Disa men uttryckt i C: $C > 1/(h|\varepsilon|)$, till exempel $C > 10^{14}$ för $\varepsilon = -10^{-10}$ och $h = 10^{-4}$. Fredrik hade ytterligare en formulering av exakt samma sak.

5. **Automatlösare:** Disa tänkte att en adaptiv steglängdskontroll (se Disas lösning) och detektering av teckenbyte (se Disas lösning) skulle kunna hjälpa. Hampus hade samma ideer. Anton föreslog samma teckenkontroll: förkasta steget och halvera h om y byter tecken.

Uppgift 5. Inte PANG

Uppgift 5. Disa Degerholm

Betrakta begynnelsevärdesproblemet

$$y' = y, \quad y(0) = N. \tag{11}$$

1. Analytisk lösning och globalt fel för RK4

Differentialekvationen har den exakta lösningen:

$$y(t) = Ne^t \implies y(2^k) = Ne^{2^k}. \tag{12}$$

För $y' = y$ gäller $y^{(v)}(\xi) = Ne^{\xi}$. Det lokala trunkeringsfelet per steg ges av:

$$\epsilon_{\text{lokal}} = -\frac{y^{(v)}(\xi)}{120}h^5 = -\frac{Ne^{\xi}}{120}h^5. \tag{13}$$

Över $M = \frac{T}{h}$ steg fram till $T = 2^k$ begränsas det globala felet av:

$$|E_M| \leq M \cdot \max_{0\leq\xi\leq T} |\epsilon_{\text{lokal}}(\xi)| \cdot e^{T-\xi} \leq \frac{T}{h}\left(\frac{Ne^T}{120}h^5\right) = \frac{N \cdot 2^k e^{2^k}}{120}h^4. \tag{14}$$

Kravet på två korrekta decimaler ($|E_M| \leq 0{,}005$) kräver:

$$\frac{N \cdot 2^k e^{2^k}}{120}h^4 \leq 0{,}005 \implies h \leq \left(\frac{0{,}9}{N \cdot 2^k}\right)^{1/4} e^{-2^{k-2}}. \tag{15}$$

Steglängden h måste alltså minskas exponentiellt, vilket gör att antalet steg $M = \frac{2^k}{h} \propto 2^{5k/4}e^{2^{k-2}}$ växer explosionsartat.

2. Datorns begränsningar (Varför datorn ger upp)

I standard flyttalsformat (`float64`, $\epsilon_{\text{mach}} \approx 2{,}22 \cdot 10^{-16}$) stoppas beräkningen av två skilda orsaker:

1. **Förlust av precision ($k \geq 5$):** Avståndet mellan två på varandra följande representerbara flyttal vid storleken y är ulp $\approx \epsilon_{\text{mach}} \cdot y$. För två korrekta decimaler krävs $\epsilon_{\text{mach}} \cdot y \leq 0{,}005$:

$$y \leq \frac{0{,}005}{2{,}22 \cdot 10^{-16}} \approx 2{,}25 \cdot 10^{13} \implies 2^k \leq \ln\left(\frac{2{,}25 \cdot 10^{13}}{N}\right). \tag{16}$$

   För $N = 82$ passeras gränsen redan vid $t \approx 26{,}3$, dvs. för alla $k \geq 5$ ($t_5 = 32$). Då är avståndet mellan talen större än toleransen, vilket gör två korrekta decimaler omöjliga oavsett hur litet h väljs då $|E_M| \leq 0{,}005$ inte kan uppnås.

2. **Aritmetiskt överflöde / `overflow` ($k \geq 10$):** Det maximala talet i `float64` är $y_{\max} \approx 1{,}79 \cdot 10^{308}$. Detta överskrids då:

$$Ne^{2^k} > 1{,}79 \cdot 10^{308} \implies 2^k > \ln\left(\frac{1{,}79 \cdot 10^{308}}{82}\right) \approx 705{,}4. \tag{17}$$

   Eftersom $t_9 = 512$ och $t_{10} = 1024$, slår beräkningen i hårdvarutaket vid $k = 10$ och returnerar `inf`.

3. Resultat

k	t_k	y_exakt	y_num	Absolut fel	Status
1	2	605.9026	605.9026	1.59e-06	OK (100 steg)
2	4	4477.0483	4477.0479	3.70e-04	OK (100 steg)
3	8	244438.5549	244438.5524	2.56e-03	OK (400 steg)
4	16	728661062.6816	728661062.6779	3.79e-03	OK (6400 steg)
5	32	6.47e+15	-	-	Stopp: Maskinprecision omöjliggör 2 dec
6	64	5.11e+29	-	-	Stopp: Maskinprecision omöjliggör 2 dec
7	128	3.19e+57	-	-	Stopp: Maskinprecision omöjliggör 2 dec
8	256	1.24e+113	-	-	Stopp: Maskinprecision omöjliggör 2 dec
9	512	1.87e+224	-	-	Stopp: Maskinprecision omöjliggör 2 dec
10	1024	inf	inf	-	Krasch: Overflow i float64

Uppgift 5. Anton Fornstedt

Problemet löstes med Eulers metod, där h halverades från 0,1 tills $|y_n - 34e^T| < 0{,}005$. Det teoretiska felet fås ur $(1+h)^{T/h} = e^{(T/h)\ln(1+h)} \approx e^T(1 - Th/2)$, vilket ger

$$\text{fel}(T) \approx 34\, e^T \cdot \frac{Th}{2}.$$

k	T	h	Antal steg	y_n	Fel	Teoretiskt fel
1	2	$1{,}22 \cdot 10^{-5}$	163 840	251,22484	0,00307	0,00307
2	4	$7{,}63 \cdot 10^{-7}$	5 242 880	1 856,33427	0,00283	0,00283
3	8	$1{,}19 \cdot 10^{-8}$	671 088 640	101 352,56673	0,00483	0,00483

Det uppmätta felet stämmer med det teoretiska på 3–4 siffror. För $k = 4$ ($T = 16$) skulle det krävas $h \lesssim 2 \cdot 10^{-12}$, alltså omkring $8 \cdot 10^{12}$ steg, vilket inte är genomförbart. Dessutom skulle avrundningsfelen dominera långt innan dess.

Uppgift 5. Hampus Hillding

Med Eulers metod fick jag

- k=0: $y(2^k) = 266.39$
- k=1: $y(2^k) = 724.12$
- k=2: $y(2^k) = 5350.61$
- k=3: $y(2^k) = 292133.84$
- k=4: tog för lång tid.

Teoretiskt fel $= \frac{nMh^2}{2} = 2^k \frac{Mh}{2} = 2^k \cdot Ce^{2^k} \cdot \frac{h}{2}$ om $M = y''(2^k) = Ce^{2^k}$.
Skillnaden mellan det faktiska felet och det teoretiska felet blev ungefär 2.64e-11 för k=0, 0.000276 för k=1, 0.000255 för k=2 och 0.0244 för k=3.

Uppgift 5. Fredrik Sollenberg

Analytiskt: Begynnelsevärdesproblemet $y' = y, y(0) = N$ har uppenbart lösningen Ne^x.

Numeriskt: Uppgiften är att räkna ut $y(2^k) = Ne^{(2^k)}$ som om man inte visste om den analytiska lösningen. Jag använde mig av modulen mpmath som har godtycklig precision och dess funktion odefun() som returnerar en funktion som är en taylorapproximation av lösningskurvan med önskad precision. Värderar man denna funktion i $t = 2^k$ får man tillbaka lösningen.

Observationer m. Python: Kör man programmet fick jag snabbt ut värden för k upp till 6. 7 tog ett par sekunder och 8 tog någon minut. Jag hade igång programmet ganska länge och fick ett värde för $k = 9$. Därefter gav jag upp eftersom nästa tal skulle ta orimligt lång tid och programmet använde 18 GB av mitt RAM.

Uppgift 5. Skillnader och likheter

1. **Lös numeriskt med 2 korrekta decimaler:** Hampus gjorde Euler, Disa gjorde RK4. Både tog fram den teoretiska formeln för det totala trunkeringsfelet $|E_n|$. Anton använde Euler och nådde bara $k = 3$, vilket krävde 671 miljoner steg, medan Disa med RK4 nådde $k = 4$ med 6 400 steg. Det visar tydligt skillnaden mellan fel $\sim h$ och $\sim h^4$. Fredrik använde modulen Mpmath och fick en lösning för $k = 9$.

2. **Jämför teoretiska och numeriska felen:** Eulers metod har global konvergensordning $\mathcal{O}(h)$ medan RK4 har $\mathcal{O}(h^4)$. Disa konstaterade att RK4 för ett givet steg h ger långt fler korrekta decimaler än Euler eftersom varje steghalvering krymper felet med en faktor $2^4 = 16$ istället för bara 2. Detta förklarar varför Hampus lyckades nå 2 korrekta decimaler med Euler för $k = 3$, men för $k = 4$ exploderade beräkningstiden ($N \propto 1/h$) och tog för lång tid. Antons asymptotiska felformel $34e^T \cdot Th/2$ stämde med det uppmätta felet på 3-4 siffror.

Uppgift 6. Vad är π?

Uppgift 6. Disa Degerholm

Betrakta begynnelsevärdesproblemet

$$y'' + y = 0, \quad y(0) = 1, \quad y'(0) = 0. \tag{18}$$

1. Analytisk lösning och kvalitativ analys

Karakteristiska ekvationen $r^2 + 1 = 0 \implies r = \pm i$ ger $y(t) = c_1 \cos(t) + c_2 \sin(t)$. Begynnelsevillkoren ger $c_1 = 1, c_2 = 0$, dvs. den unika analytiska lösningen:

$$y(t) = \cos(t). \tag{19}$$

Kvalitativ definition av π (Simmons kap. 4.24): Låt $s(t)$ och $c(t)$ vara lösningarna till $y'' + y = 0$ med startvärdena $s(0) = 0, s'(0) = 1$ respektive $c(0) = 1, c'(0) = 0$.

- Eftersom $(y')'' + y' = 0$ gäller $s'(t) = c(t)$ och $c'(t) = -s(t)$.
- Energin är konstant bevarad:

$$\frac{d}{dt}\big(s(t)^2 + c(t)^2\big) = 2s(t)s'(t) + 2c(t)c'(t) = 2s(t)c(t) - 2c(t)s(t) = 0, \tag{20}$$

  vilket med startvärdena ger $s(t)^2 + c(t)^2 \equiv 1$.

- Kurvan $s(t)$ stiger från origo med avtagande lutning ($s'' = -s < 0$) och når en topp vid $s = 1$ där $s' = c = 0$. Den första positiva skärningspunkten för $s(t) = 0$ definieras som π. Av symmetri nås toppen vid $t^* = \pi/2$ där hastigheten är noll:

$$c(t^*) = 0 \implies t^* = \frac{\pi}{2} \iff \pi = 2t^*. \tag{21}$$

- Enligt separationssatsen (Simmons kap. 4.24/4.25) alternerar nollställena för $s(t)$ och $c(t)$ strikt, varför nollstället t^* är isolerat och unikt på $(0, \pi)$.

2. Omskrivning till första ordningens system

Med $v(t) = y'(t)$ fås $v'(t) = y''(t) = -y(t)$:

$$\begin{cases} y'(t) = v(t) \\ v'(t) = -y(t) \end{cases} \qquad \text{med} \quad \begin{pmatrix} y(0) \\ v(0) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}. \tag{22}$$

3. Numerisk skattning av första nollstället $t^* = \pi/2$

När lösaren identifierar ett steg där $y_n > 0$ och $y_{n+1} \leq 0$ bestäms t^* via ett linjärt Newton-steg längs tangenten:

$$t^* \approx t_n - \frac{y_n}{y_n'} = t_n - \frac{y_n}{v_n} \implies \pi \approx 2t^*. \tag{23}$$

- **Eulers metod (Varför datorn smälter):** Att nå maskinprecision (~ 16 siffror) med Eulers metod kräver $N \sim 10^{16}$ steg eftersom metodens globala trunkeringsfel är $\mathcal{O}(h)$, vilket för 16 siffror kräver en steglängd $h \approx 10^{-16}$ och därmed $N = T/h \approx T/10^{-16} \sim 10^{16}$ beräkningssteg. Detta är beräkningsmässigt orimligt och havererar i flyttalsaritmetiken då varje operation medför ett relativt avrundningsfel $\sim \epsilon_{\text{mach}} \approx 2{,}22 \cdot 10^{-16}$, vilket över alla steg ackumuleras som $N \cdot \epsilon_{\text{mach}} = (T/h)\epsilon_{\text{mach}} \propto \frac{\epsilon_{\text{mach}}}{h}$, en felterm som exploderar då $h \to 0$.
- **RK4:** Med global konvergensordning $\mathcal{O}(h^4)$ reduceras felet med en faktor 16 per halvering av h, vilket ger hög noggrannhet med måttligt antal steg (t.ex. $h = 10^{-3}$).

4. Skattning med 20 korrekta decimaler

Standard flyttal (`float64`) har 53 bitars mantissa ($\epsilon_{\text{mach}} \approx 2{,}22 \cdot 10^{-16}$) och är begränsat till $\sim$ 15–17 decimaler. Att nå 20 säkra decimaler löses genom:

1. Godtycklig flyttalsprecision (`mpmath` satt till 40 siffrors intern precision som säkerhetsmarginal mot ackumulerade avrundningsfel).

2. **Taylor-polynom och Newtons metod:** För att undvika att ackumulera trunkerings- och fasfel från tusentals små tidssteg med stegmetoder (som RK4), utnyttjas att ODE:ns derivator är kända analytiskt vid $t = 0$. Hela lösningskurvan fram till nollstället ersätts då med ett enda analytiskt Taylor-polynom:

$$y_M(t) = \sum_{k=0}^{M} \frac{(-1)^k}{(2k)!} t^{2k}, \quad y'_M(t) = \sum_{k=1}^{M} \frac{(-1)^k}{(2k-1)!} t^{2k-1}. \tag{24}$$

Genom att välja $M = 16$ (polynomgrad 32) begränsas seriens trunkeringsfel av resttermen $|R_{16}(t)| \leq \frac{t^{34}}{34!} < 10^{-38}$ vid $t \approx 1{,}57$, vilket gör approximationsfelet helt försumbart jämfört med kravet på 10^{-20}. Nollstället $t^* = \pi/2$ beräknas därefter snabbt och med kvadratisk konvergens via Newtons metod direkt på polynomet:

$$t \leftarrow t - \frac{y_M(t)}{y'_M(t)}. \tag{25}$$

5. Skattning av π nära $t \approx 10^3$

Nollställena är $t_k = \frac{\pi}{2} + k\pi = \left(k + \frac{1}{2}\right)\pi$. Närmast $T = 1000$ ges av:

$$k = \text{round}\left(\frac{1000 - \pi/2}{\pi}\right) = 318 \implies t_{318} = 318{,}5\pi \approx 1000{,}597. \tag{26}$$

Skattningen av π ur det beräknade nollstället $\hat{t}_{318}$ sker via $\hat{\pi} = \frac{\hat{t}_{318}}{318{,}5}$.

Jämförelse mellan teoretiskt fel och beräkningsfel:

- **Teoretisk dämpning:** Eftersom nollställena ges av $t_k = (k + 0{,}5)\pi$ skattas π via $\hat{\pi} = \frac{\hat{t}_k}{k+0{,}5}$. För $k = 318$ ger ett numeriskt beräkningsfel $\delta t = |\hat{t}_{318} - t_{318}|$ därmed direkt felet i π:

$$\delta\pi = \left|\frac{\hat{t}_{318}}{318{,}5} - \frac{t_{318}}{318{,}5}\right| = \frac{\delta t}{318{,}5}. \tag{27}$$

  Divisionen med $k + 0{,}5$ dämpar alltså felet i π med en faktor drygt 300.

- **Numerisk felförstoring:** Integrationstiden $t \approx 1000$ är över 600 gånger längre än $t \approx \pi/2$. För en enstegsmetod (som RK4) ackumuleras fasfelet linjärt med integrationstiden, vilket ger över 600 gånger större fasdrift. Dessutom genererar de hundratusentals beräkningsstegen betydligt större ackumulerade flyttalsfel.

- **Slutsats:** Resultaten visar att det slutliga skattningsfelet inte nödvändigtvis ökar med integrationstiden. För RK4 är felet vid beräkningen av det första nollstället $3{,}37 \cdot 10^{-10}$, medan skattningen av π nära $t = 1000$ ger felet $5{,}25 \cdot 10^{-11}$. Det senare felet är ungefär 6,4 gånger mindre.

  Detta innebär dock inte att den långa integrationen generellt är mer noggrann. Det slutliga felet påverkas både av det ackumulerade integrationsfelet och av hur dessa fel fortplantas genom skattningsformeln. Resultaten visar därför att skattningen nära $t = 1000$ i just denna beräkning har bättre precision, men de räcker inte för att dra slutsatsen att lång integration generellt ger bättre eller sämre precision.

```
Euler pi: 3.141593 | Fel: 1.05e-08
RK4 pi:   3.1415926539 | Fel: 3.37e-10
Taylor 20 dec: 3.14159265358979323846
Rot t*: 1000.5973 (k = 318)
Skattat pi: 3.1415926535 | Fel: 5.25e-11
```

Uppgift 6. Anton Fornstedt

Den analytiska lösningen är $y = \cos t$, så första nollstället är $\pi/2$. Systemet $u' = v$, $v' = -u$ löstes tills u bytte tecken, och nollstället skattades därefter. Att räkna tiden som $t = t + h$ gav ackumulerade avrundningsfel (felet stannade vid ca 10^{-10}), så tiden räknades i stället som $t = i \cdot h$.

h	Euler + lin. interp.	RK4 + lin. interp.	RK4 + Newton
10^{-1}	$1{,}02 \cdot 10^{-2}$	$2{,}61 \cdot 10^{-5}$	$2{,}66 \cdot 10^{-6}$
10^{-2}	$1{,}05 \cdot 10^{-4}$	$2{,}08 \cdot 10^{-8}$	$2{,}62 \cdot 10^{-10}$
10^{-3}	$1{,}05 \cdot 10^{-6}$	$3{,}20 \cdot 10^{-11}$	$2{,}62 \cdot 10^{-14}$
10^{-4}	$1{,}05 \cdot 10^{-8}$	$1{,}09 \cdot 10^{-14}$	$2{,}62 \cdot 10^{-18}$
10^{-5}	$1{,}05 \cdot 10^{-10}$	$2{,}06 \cdot 10^{-17}$	$2{,}62 \cdot 10^{-22}$
10^{-6}	$9{,}64 \cdot 10^{-13}$	$2{,}54 \cdot 10^{-20}$	–
10^{-7}	$9{,}19 \cdot 10^{-14}$	–	–

Euler ger fel $\approx \pi h^2/3$ och når som bäst 12 korrekta decimaler. Med RK4 begränsas resultatet av den linjära interpolationen ($\sim h^3$). Genom att i stället förfina nollstället med Newton, $\delta = -u/v$, följt av ett RK4-steg av längd δ, fås felet $\approx \pi h^4/120$. Med `mpmath` (40 siffror) gav $h = 10^{-5}$ 21 korrekta decimaler.

Nollstället nära $t = 1000$. Det ligger vid $t^* = 318{,}5\pi$, så $\pi \approx t^*/318{,}5$. Felet i π blev detsamma som för första nollstället ($2{,}62 \cdot 10^{-10}$ för $h = 0{,}01$). Felet i t^* växer linjärt med tiden men delas med 318,5. Det som blir svårare är kostnaden: cirka 640 gånger fler steg.

Uppgift 6. Hampus Hillding

Analytiskt fick jag lösningen $y(t) = cos(t)$ som har ett första nollställe vid $t = \pi/2$.
Numeriskt:
Jag skrev om ekvationen till ett system av förstagrads-ODE:er. Med Eulers metod på systemet fick jag 9 korrekta siffror av π. För att få fler siffror använde jag Runge-Kutta av ordning 4. Jag stegade tills mitt y bytte tecken. Då använde jag Newtons metod för att komma ännu närmare lösningen som jag visste fanns inom det sista steget i RK4. Med steglängd $h = 10^{-3}$ fick jag då 13 korrekta siffror av π, och med steglängd $h = 10^{-5}$ fick jag 22 korrekta siffror.
Rot nära t=1000:
Jag modifierade koden för att hitta roten närmast $t = 1000$, dvs. $t = 637\pi/2$. Med $h = 10^{-3}$ fick jag lika många korrekta siffror som tidigare, dvs. 13. I båda fallen fick jag också ett fel $\approx 2.6 \cdot 10^{-14}$. Enligt teorin borde cirka 637 gånger så många steg ge ungefär 637 gånger så stort fel i värdet på t men eftersom vi multiplicerar roten med $\frac{2}{637}$ istället för bara 2 för att få värdet på π tar dessa effekter ut varandra i skattningen av π. Den huvudsakliga svårigheten blir att det tar längre tid för programmet att köras.

Uppgift 6. Fredrik Sollenberg

Analytiskt: Karakteristiska ekvationen och begynnelsevillkoren ger $y = \frac{1}{2}e^{(it)} + \frac{1}{2}e^{(-it)}$ vilket är identiskt med Eulers formel-definitionen av cosinus. För de numeriska beräkningarna skrevs ekvationen om som ett system av första ordningens ODE:er.

Numeriskt: Om vi vet något om cosinus så är det att första nollstället är vid $\frac{\pi}{2}$. Genom att beräkna nollstället numeriskt kan vi beräkna värdet på π. Vi hittar nollstället och multiplicerar med 2. Vi skulle kunna använda vilket nollställe som helst men det tidigaste är bäst för att ju längre bort man kommer från $t = 0$ desto större fasförskjutning får man. En metod som Euler frmåt eller RK4 kommer att genomföra en period av cosinusfunktionen efter tiden $t = \pi + \delta$ för något fel δ. Efter många svängningar har felen byggts upp. Även om vi då hittar det numeriska nollstället med hög precision kommer det att vara förskjutet en aning och ge ett sämre svar. Denna förskjutning ökar linjärt med antal svängningar.

Observationer m. Python: Jag använde tre metoder, Euler framåt (den sög), scipys funktion solve_ivp med RK ordning 8, och mpmath med taylor. Solve_ivp räknar ut så bra det går (≈ 14 decimaler) men den stödjer inte godtycklig precision av sig själv. Mpmath räknade ut perfekt, bara sådär. Av praktiska skäl användes solve_ivp för uppgiften att beräkna π vid $t = 1000$ och intressant nog var felet tydligt vilket jag skyller på fasföskjutningen. Gällande uppgiften att beräkna godtyckligt många decimaler av pi användes mpmath. Jag fick 10 miljoner korrekta decimaler på några få minuter. Den enda egentliga begränsningen var tiden det tog; mitt RAM märkte knappt av det.

Uppgift 6. Skillnader och likheter

1. **Analytisk lösning:** Alla i gruppen fick att $y(t) = cost(t)$.

2. **Hur många korrekta siffror kan man få innan datorn smälter:** Att datorn smälter beror på två faktorer: beräkningstiden exploderar ($N \propto 1/h$) och flyttalsavrundningar ($\epsilon_{\text{mach}} \approx 2{,}2 \cdot 10^{-16}$) tar över då felet växer som $\mathcal{O}(\epsilon_{\text{mach}}/h)$. Med standard dubbelprecision (`float64`) nådde Disa ca 11–12 korrekta siffror med RK4 ($h \sim 10^{-3}$ till 10^{-4} kombinerat med Newtons metod för nollstället $t^* = \pi/2$), medan hårdvarugränsen på 53 bitars mantissa förhindrar fler än ca 15–16 siffror. För Euler nådde Hampus ca 9 siffror innan tidsåtgången och avrundningsfelen satte stopp. Anton visade att begränsningen i Euler dels berodde på fasfelet $\approx \pi h^2/3$, dels på ackumulerade avrundningar från tidsuppdateringen $t = t + h$, vilket förbättrades till 12 decimaler med $t = i \cdot h$. För att nå 20 korrekta decimaler eliminerades flyttalsgränsen med godtycklig precision i `mpmath` som använde ett Taylor-polynom av grad 30 ($M \geq 15$) kombinerat med Newtons metod, medan Anton använde RK4 med $h = 10^{-5}$ och Newton-förfining.

3. **Hur mycket svårare blir det med skifte:** Disa visade att nollstället närmast $T = 1000$ ges av $t_k = \pi/2 + k\pi$, vilket ger $k = 318$ och $t_{318} = 318{,}5\pi \approx 1000{,}597$[cite: 3]. Ur formeln $\pi = \hat{t}_{318}/318{,}5$ dämpas felet i nollstället med en teoretisk faktor 318,5[cite: 3]. Däremot växer metodens fasfel och trunkeringsfel linjärt med integrationstiden ($\sim t \cdot h^p$), vilket vid $t \approx 1000$ är en faktor drygt 600 större än vid $t = \pi/2$, samtidigt som det ackumulerade avrundningsfelet ökar med antalet steg. Därmed tar den numeriska feltillväxten överhanden och gör skattningen vid 10^3 betydligt svårare. Anton fick däremot samma fel i π som för första nollstället ($2{,}62 \cdot 10^{-10}$ för $h = 0{,}01$). Felet i t^* växer linjärt med antalet svängningar, men delas med 318,5, så effekterna tar ut varandra. Det som blir svårare är kostnaden (cirka 640 gånger fler steg), inte precisionen. Hampus fick också ungefär sammma fel i sin skattnig av π vid båda nollställena.

Uppgift 7. Newton!

Uppgift 7. Disa Degerholm

1. Rörrelseekvationer och bevarandelagar

Betrakta en stjärna fix i origo med massan M, där gravitationsparametern normeras till $GM = 1$. En planet med massa m har lägesvektorn $\mathbf{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$. Newtons gravitations- och rörelselagar ger accelerationen:

$$\mathbf{r}''(t) = -\frac{\mathbf{r}}{\|\mathbf{r}\|^3} \iff \begin{cases} x''(t) = -\dfrac{x}{(x^2+y^2)^{3/2}} \\ y''(t) = -\dfrac{y}{(x^2+y^2)^{3/2}} \end{cases} \tag{28}$$

Genom att införa hastigheterna $v_x = x'$ och $v_y = y'$ formuleras problemet som fyra kopplade första ordningens differentialekvationer:

$$\frac{d}{dt}\begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ -x/(x^2+y^2)^{3/2} \\ -y/(x^2+y^2)^{3/2} \end{pmatrix}. \tag{29}$$

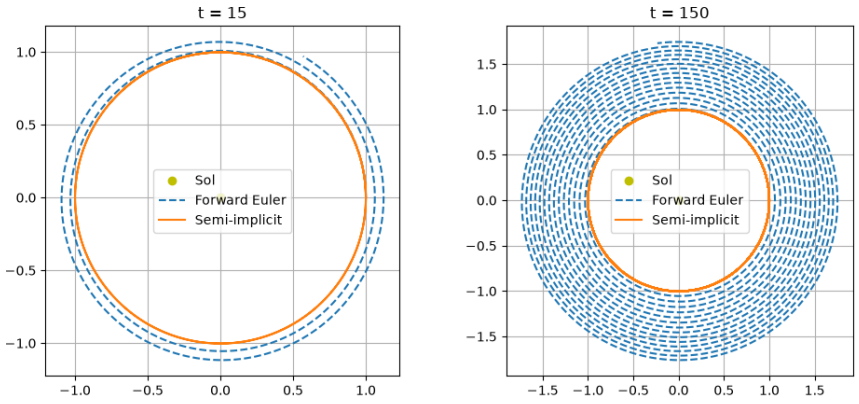
Systemet har två analytiska rörelsekonstanter:

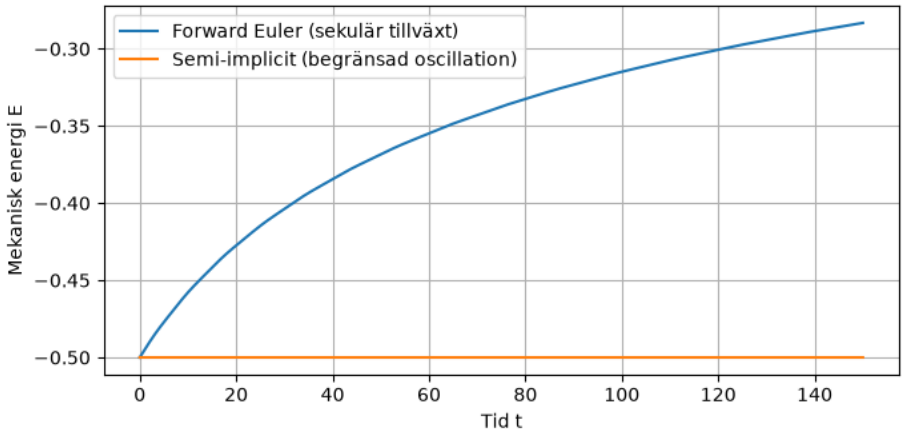
- **Total mekanisk energi per massenhet (E):**

$$E = \frac{1}{2}(v_x^2 + v_y^2) - \frac{1}{\sqrt{x^2+y^2}}, \quad \text{där } \frac{dE}{dt} = 0. \tag{30}$$

- **Rörelsemängdsmoment per massenhet (L):**

$$L = xv_y - yv_x, \quad \text{där } \frac{dL}{dt} = 0. \tag{31}$$





Uppgift 7. Anton Fornstedt

Härledning. Med $GM = 1$ och $r = \sqrt{x^2 + y^2}$ ger Newtons gravitationslag

$$x'' = -\frac{x}{r^3}, \qquad y'' = -\frac{y}{r^3},$$

som skrevs om till ett första ordningens system i (x, y, v_x, v_y). Med start i $(1, 0)$ ger $v_y = 1$ en cirkelbana, eftersom cirkelhastigheten är $\sqrt{GM/r} = 1$. Flykthastigheten är $\sqrt{2GM/r} = \sqrt{2}$. Energin $E = \frac{1}{2}(v_x^2 + v_y^2) - 1/r$ ska vara konstant.

Explicit mot semi-implicit Euler. Med explicit Euler spiralar planeten utåt och energin växer stadigt. I semi-implicit (symplektisk) Euler uppdateras först hastigheten och sedan positionen med den nya hastigheten. Då förblir banan sluten och energin håller sig nära sitt startvärde ($h = 0{,}01$, $T = 10\,000$):

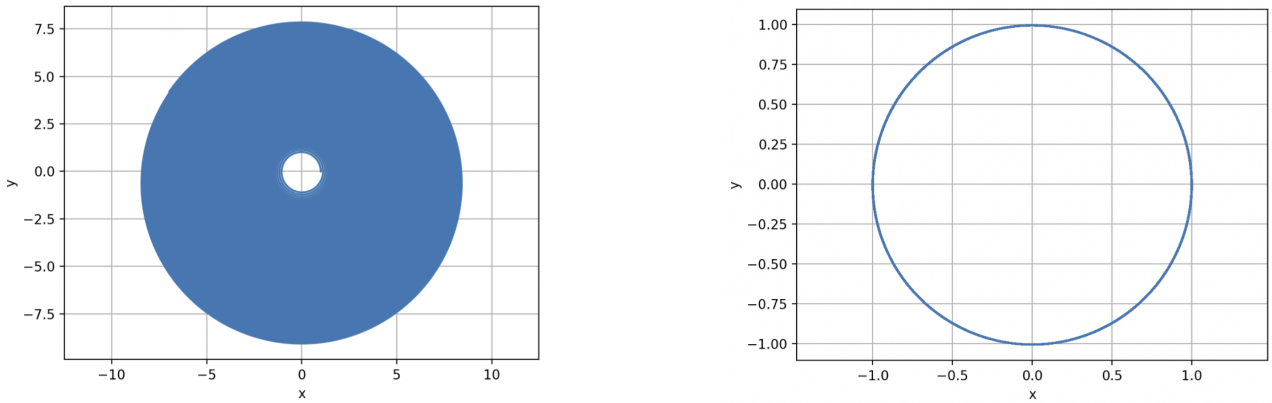
Metod	$v_y(0)$	$E(0)$	$E(10\,000)$
Explicit Euler	1	$-0{,}5$	$-0{,}0590$
Semi-implicit Euler	1	$-0{,}5$	$-0{,}49995$
Semi-implicit Euler	1,2	$-0{,}28$	$-0{,}2801$

Explicit Euler förlorade alltså nästan all bindningsenergi trots litet h, medan felet med semi-implicit Euler bara oscillerar och aldrig växer.

Uppgift 7. Hampus Hillding

Systemets ODE: Låt $\mathbf{r}(t) = (x(t), y(t))$. Gravitationskraften från solen på fås av $F = G\frac{m_{jord}m_{sol}}{r^2}$ Vi väljer m_{sol} så att $Gm_{sol} = 1$. Eftersom detta är den enda kraften som verkar på jorden får vi via Newtons andra lag att

$$\ddot{\mathbf{r}} = -\frac{1}{r^2}\mathbf{e_r} = -\frac{1}{r^2} \cdot \frac{\mathbf{r}}{r}$$



Figur 2: Explicit Euler (vänster) och semi-implicit Euler (höger), $h = 0{,}01$, $T = 10\,000$.

$$r = \sqrt{x^2 + y^2} \text{ ger } \ddot{\mathbf{r}} = -\frac{\mathbf{r}}{(x^2+y^2)^{\frac{3}{2}}}$$

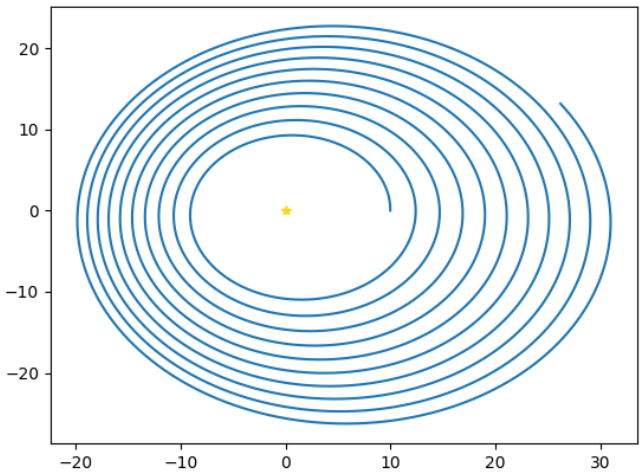
Detta kan alternativt skrivas som systemet

$$\begin{cases} \ddot{x} = -\frac{x}{(x^2+y^2)^{3/2}} \\ \ddot{y} = -\frac{y}{(x^2+y^2)^{3/2}} \end{cases}$$

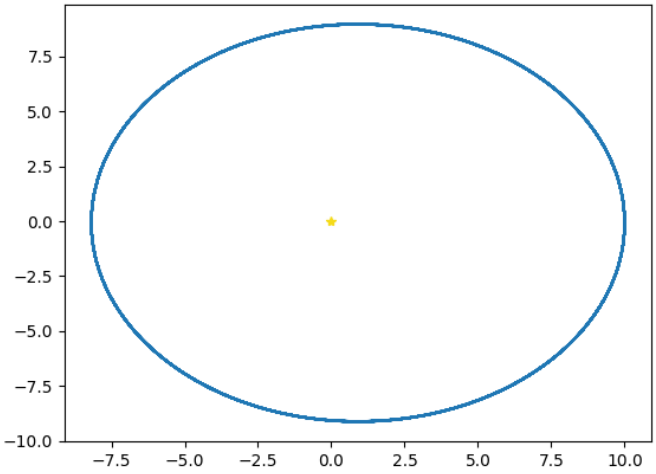
Simulera: För att använda Eulers metod skrev jag om systemet som ett system av första ordningens differentialekvationer:

$$\begin{cases} \dot{x} = u \\ \dot{u} = -\frac{x}{(x^2+y^2)^{3/2}} \\ \dot{y} = v \\ \dot{v} = -\frac{y}{(x^2+y^2)^{3/2}} \end{cases}$$

Med vanliga Eulermetoden rör sig jorden i en utåtgående spiral. Detta är en dålig modell av verkligheten eftersom energin inte bevaras.



Med semi-implicit Euler får jag istället en sluten elliptisk bana som bevarar energin.



Uppgift 7. Fredrik Sollenberg

Analytiskt: Vi börjar med att anta att solen inte kan flytta sig från origo. Då behöver vi bara beskriva jordens rörelse med D.E:er. Vi utgår från Newtons gravitationslag.

$$\ddot{\mathbf{r}} = -\frac{GM}{r^2}\frac{\mathbf{r}}{r}$$

vilket med komposantuppdelning av $\mathbf{r} = (x, y)$ blir

$$\begin{cases} a_x = -\frac{GMx}{r^3} \\ a_y = -\frac{GMy}{r^3} \end{cases}$$

som vi kan räkna med

Numeriskt: Om vi plottar fasplanet för det här begynnelsevärdesproblemet får vi den simulerade planetbanan. När man simulerar planetbanor är det viktigt att man använder en symplektisk metod på D.E:n eftersom annars åker jorden in i solen och alla dör. Jag blev varse. Sen efter att jag implementerade den symplektiska metoden semiimplicit Euler så dog inte alla.

Observationer m. Python: Jorden snurrar runt solen.

Uppgift 7. Skillnader och likheter

1. **Härled systemets ODE:** Alla gjorde samma förutom att Fredrik gjorde ett andra ordningens system medan Disa, Anton och Hampus gjorde ett första ordningens system - men båda kan härledas från varandra.
2. **Simulera för olika initialvärden under kort och lång tid:** Disa upptäckte att för lång tid blir det en instabil spiral utåt och för kort tid så syns inte detta. Detta är på grund av att energin ökar på grund av den numeriska härledningen (se Disas svar). Anton fick samma utåtgående spiral med explicit Euler: på $T = 10\,000$ ökade energin från $-0{,}5$ till $-0{,}059$ med $h = 0{,}01$. (Kanske kunde förutspåts från sista delen på uppgift 6??)
3. **Använd Semi implcit Euler för att återställa ordningen i systemet:** Check, det gick bra efterosm att metoden är symplektisk och därför bevarar jordens energi i fasplanet.

Uppgift 8. Newton revisited (something for nothing)

Uppgift 8. Disa Degerholm

1. Modellering och rörelseekvationer i trekropparsproblemet

Systemet modelleras som ett planärt, cirkulärt begränsat trekropparsproblem bestående av Solen fix i origo ($M_S = 1$), Jupiter ($M_J = 0{,}08$) i cirkulär bana, samt en rymdsond med försumbar massa ($m \to 0$). Jordens gravitationella påverkan försummas då dess influensradie är försumbar i förhållande till solens gravitationsfält.

Astronomiska enheter väljs så att $G = 1$, längdenheten är 1 AU och $GM_S = 1$. Keplers tredje lag ger Jupiters vinkelhastighet vid banradien $R_J = 5{,}2$ AU:

$$\omega_J = \sqrt{\frac{GM_S}{R_J^3}} = R_J^{-3/2} = 5{,}2^{-3/2} \approx 0{,}0843 \text{ rad/tidsenhet}. \tag{32}$$

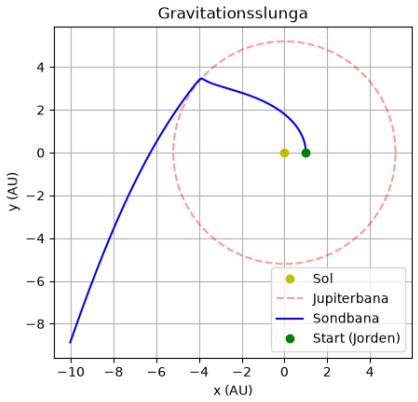
Jupiters banvektor ges av:

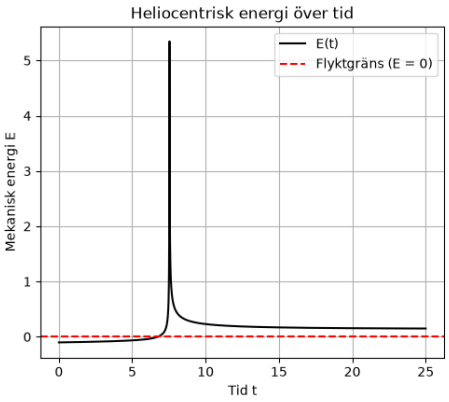
$$\mathbf{r}_J(t) = \begin{pmatrix} x_J(t) \\ y_J(t) \end{pmatrix} = \begin{pmatrix} R_J \cos(\omega_J t + \phi_0) \\ R_J \sin(\omega_J t + \phi_0) \end{pmatrix}, \tag{33}$$

där fasvinkeln ϕ_0 anger Jupiters position vid $t = 0$.

Superposition av Newtons gravitationslag för Solen och Jupiter ger kraften per massenhet på sonden vid $\mathbf{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$:

$$\mathbf{r}''(t) = -\frac{\mathbf{r}}{\|\mathbf{r}\|^3} - M_J \frac{\mathbf{r} - \mathbf{r}_J(t)}{\|\mathbf{r} - \mathbf{r}_J(t)\|^3}. \tag{34}$$






```
Energi vid start:        -0.1022
Energi efter passage:     0.1492
Resultat: Flykt (E > 0)
```

Med hastigheterna $v_x = x'$ och $v_y = y'$ formuleras rörelsen som fyra kopplade första ordningens differentialekvationer:

$$\frac{d}{dt}\begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ -\dfrac{x}{(x^2+y^2)^{3/2}} - M_J \dfrac{x - x_J}{\left((x-x_J)^2 + (y-y_J)^2\right)^{3/2}} \\ -\dfrac{y}{(x^2+y^2)^{3/2}} - M_J \dfrac{y - y_J}{\left((x-x_J)^2 + (y-y_J)^2\right)^{3/2}} \end{pmatrix}. \tag{35}$$

2. Flyktvillkor och energiprincip

Sondens specifika mekaniska energi relativt Solen definieras av:

$$E_{\text{helio}}(t) = \frac{1}{2}\|\mathbf{v}(t)\|^2 - \frac{1}{\|\mathbf{r}(t)\|}. \tag{36}$$

Banans geometri bestäms av energins tecken:

- **Bunden bana ($E < 0$):** Flykthastigheten vid $r = 1$ AU ges av $v_{\text{esc}} = \sqrt{2} \approx 1{,}414$. Startfarten sätts till $v_0 = 1{,}34 < \sqrt{2}$, vilket ger en bunden startellips med $E_0 = \frac{1}{2}(1{,}34)^2 - 1 = -0{,}1022 < 0$. Sonden kan därmed inte lämna solsystemet på egen hand.
- **Hyperbolisk flykt ($E > 0$):** Efter närpassagen vid en tidpunkt t_{post} där sonden avlägsnat sig från Jupiters närfält gäller:

  $$E_{\text{post}} = \frac{1}{2}\|\mathbf{v}(t_{\text{post}})\|^2 - \frac{1}{\|\mathbf{r}(t_{\text{post}})\|} > 0. \tag{37}$$

  Ett positivt energivärde ($E_{\text{post}} > 0$) garanterar en öppen hyperbolisk flyktbana med asymptotisk resthastighet vid oändligheten.

3. Gravitationsslunga (Gravitational Slingshot)

En gravitationsslunga fungerar som en elastisk stöt där sonden lånar rörelseenergi av planeten.

1. Relativ rörelse kring Jupiter Låt sondens hastighet i solsystemet före och efter mötet vara $\mathbf{v}_{\text{in}}$ respektive $\mathbf{v}_{\text{ut}}$, och Jupiters banhastighet vara $\mathbf{v}_J$. I Jupiters eget vilosystem är sondens relativa hastigheter:

$$\mathbf{v}_{\text{rel, in}} = \mathbf{v}_{\text{in}} - \mathbf{v}_J, \qquad \mathbf{v}_{\text{rel, ut}} = \mathbf{v}_{\text{ut}} - \mathbf{v}_J. \tag{38}$$

Eftersom gravitationsfältet kring Jupiter är konservativt ändras inte farten under passagen, utan banan vrids bara en vinkel δ:

$$\|\mathbf{v}_{\text{rel, ut}}\| = \|\mathbf{v}_{\text{rel, in}}\| = v_{\text{rel}}. \tag{39}$$

2. Hastighetsökning i solsystemet När sonden lämnar Jupiter transformeras hastigheten tillbaka till solens koordinatsystem genom att lägga till $\mathbf{v}_J$:

$$\mathbf{v}_{\text{ut}} = \mathbf{v}_J + \mathbf{v}_{\text{rel, ut}}. \tag{40}$$

Sondens nya rörelseenergi per massenhet beräknas via skalärprodukten:

$$\frac{1}{2}\|\mathbf{v}_{\text{ut}}\|^2 = \frac{1}{2}\|\mathbf{v}_J + \mathbf{v}_{\text{rel, ut}}\|^2 = \frac{1}{2}\|\mathbf{v}_J\|^2 + \frac{1}{2}v_{\text{rel}}^2 + \mathbf{v}_J \cdot \mathbf{v}_{\text{rel, ut}}. \tag{41}$$

Termen $\mathbf{v}_J \cdot \mathbf{v}_{\text{rel, ut}} = \|\mathbf{v}_J\| v_{\text{rel}} \cos\theta$ (där θ är vinkeln mellan $\mathbf{v}_J$ och $\mathbf{v}_{\text{rel, ut}}$) avgör energiändringen:

- Passerar sonden framför planeten pekar uthastigheten mot planetens rörelseriktning ($\cos\theta < 0$), vilket bromsar sonden.
- Passerar sonden bakom planeten vinklas uthastigheten med planetens rörelseriktning ($\cos\theta > 0$), vilket ger ett positivt tillskott av rörelseenergi.

3. Numeriskt resultat Med startvinkeln $\phi_0 = 1{,}78$ rad styrs sonden så att den passerar strax bakom Jupiter. Därmed ökar den specifika energin från en bunden bana vid start till en permanent flyktbana:

$$E_0 \approx -0{,}10 < 0 \implies E_{\text{post}} \approx +0{,}14 > 0. \tag{42}$$

Eftersom $E_{\text{post}} > 0$ är banan en öppen hyperbel som garanterar att skeppet lämnar solsystemet.

Uppgift 8. Anton Fornstedt

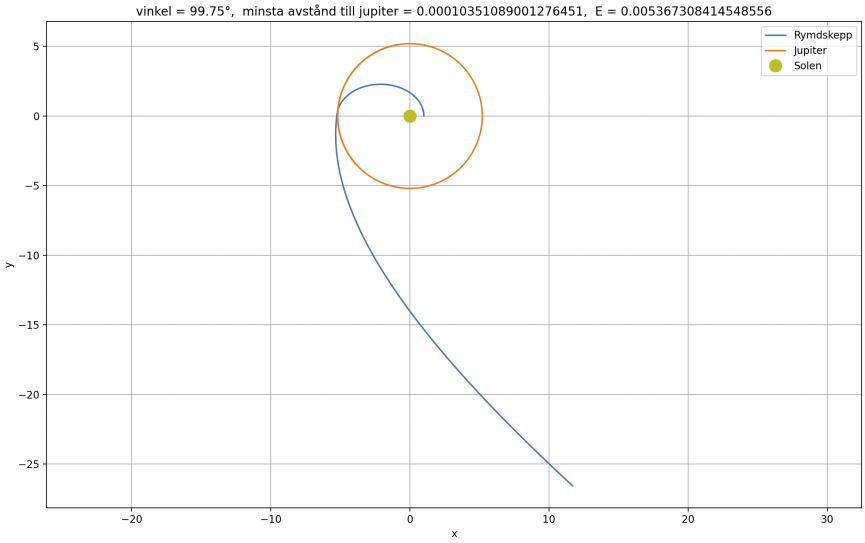
Modell. Solen ligger i origo ($GM = 1$). Jupiter går runt solen i en cirkel med radien 5,2, vinkelhastigheten $\omega = 5{,}2^{-3/2} \approx 0{,}0843$ och massan $1/1000$. Jupiter startar i vinkeln θ. Skeppet startar i $(1, 0)$ med hastigheten $(0,\, 1 + \Delta v)$ och dras av både solen och Jupiter:

$$\mathbf{r}'' = -\frac{\mathbf{r}}{r^3} - \frac{1}{1000}\,\frac{\mathbf{r} - \mathbf{r}_J(t)}{|\mathbf{r} - \mathbf{r}_J(t)|^3}.$$

Jordens dragning togs inte med. Systemet löstes med semi-implicit Euler, som i uppgift 7. Som Δv användes den fart som behövs för att nå ut till Jupiters bana (Hohmann), $\Delta v = \sqrt{2 \cdot 5{,}2/6{,}2} - 1 \approx 0{,}295$.

Resultat. Genom att testa olika startvinklar med $h = 10^{-3}$ hittades $\theta = 99{,}75°$. Sedan kördes samma vinkel med mindre h:

h	Minsta avstånd till Jupiters centrum	E
10^{-3}	0,00102	+0,0624
10^{-4}	0,000584	+0,00539
10^{-5}	0,000592	+0,00549



Figur 3: Skeppets bana med $\theta = 99{,}75°$ och $h = 10^{-4}$.

Med $h = 10^{-3}$ blir resultatet fel, men för $h = 10^{-4}$ och 10^{-5} blir svaren nästan lika, så de går att lita på. Minsta avståndet ($\approx 0{,}00059$) är större än Jupiters radie ($\approx 0{,}00048$), så skeppet passerar ungefär 15 000 km ovanför ytan och krockar inte. Eftersom $E > 0$ lämnar skeppet solsystemet.

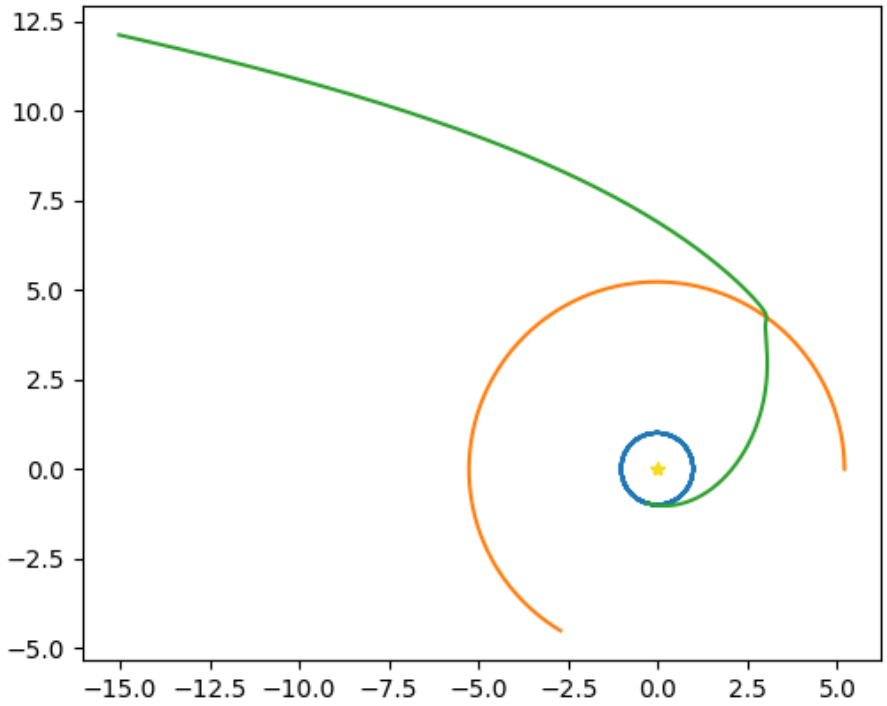
Uppgift 8. Hampus Hillding

Jag valde solens massa till $m_{sol} = 1/G$ som sist och valde jordens massa till $m_e = m_{sol}/333000$ (vilket stämmer överens ungefär med verkligheten) och jupiters massa till $m_j = m_{sol}/100$ (vilket är ca. 10 gånger tyngre än verkligheten, för lättare slingshot). Jag lät jordens bana ha radie 1 och jupiters bana ha radie 5.2. Jag lät de två planeternas banor endast bero på dragningskraften till solen och inte mellan varandra. I och med detta kunde jag återanvända ekvationssystemet från förra uppgiften för att beskriva båda planeternas respektive banor. För rymdskeppet lät jag dess bana påverkas av gravitationen från alla de tre övriga kropparna. Systemet som beskriver skeppets rörelse blir då

$$\begin{cases} \ddot{x_s} = -\frac{x_s}{(x_s^2+y_s^2)^{3/2}} + \frac{m_e(x_e-x_s)}{((x_e-x_s)^2+(y_e-y_s)^2)^{3/2}} + \frac{m_j(x_j-x_s)}{((x_j-x_s)^2+(y_j-y_s)^2)^{3/2}} \\ \ddot{y_s} = -\frac{y_s}{(x_s^2+y_s^2)^{3/2}} + \frac{m_e(y_e-y_s)}{((x_e-x_s)^2+(y_e-y_s)^2)^{3/2}} + \frac{m_j(y_j-y_s)}{((x_j-x_s)^2+(y_j-y_s)^2)^{3/2}} \end{cases}$$

Sedan använde jag semi-implicit Euler för att stega fram alla objektens banor. För att hitta rätt startpositioner för att få en slingshot lät jag programmet testa att starta med jorden i olika vinklar och tala om för vilka vinklar skeppet

passerade utanför ett visst avstånd från solen (som inte hade kunnat nås i vanliga fall). Jag lät skeppet starta på samma position som jorden men med en hastighet som gjorde att skeppets bana precis kunde korsa jupiters bana. Jupiters startläge hade vinkel noll hela tiden. Med en steglängd $h = 0.05$ hittade jag en 'gravitational slingshot' då jordens (och skeppets) startvinkel var 4.4 radianer. För att försäkra mig om att skeppet kommer att lämna solsystemet beräknade jag banans energi $E = T + V = \frac{1}{2}(\dot{x}^2 + \dot{y}^2) - \frac{1}{\sqrt{x^2+y^2}}$. (Egentligen ska allt multipliceras med massan, men vi kan ignorera den eftersom den alltid är positiv.) Jag såg då att energin var positiv, vilket innebär att skeppet lämnar solsystemet. Vid början av simuleringen är energin negativ.



Uppgift 8. Fredrik Sollenberg

I den här uppgiften tog jag en annan approach. I Python gjorde jag en klass med planeter med attribut som position och hastighet. Sen i en egen klass för själva simulationen beräknade jag gravitationen mellan alla planerterna som vektorer sen utförde jag semiimplicita Euler-metoden på systemts tillstånd och plottade.

För att göra en gravitationsslunga måste man designa ett begynnelsetillstånd som gör att ett rymdskepp skickas ut från jorden och närmar sig jupiter i sidan av dess bana och flyger in precis bakom planeten. Om gravitationsslungan har gett raketen tillräckligt med energi för att lämna solsystemet kommer dess mekaniska energi (E) att vara större än 0. E beräknas med formeln

$$E = T + V = \frac{1}{2}mv^2 + \sum_i \frac{GM_i m}{r_i}$$

Uppgift 8. Skillnader och likheter

1. **Utforska numeriskt för att hitta olika planetbanor så att systemet stämmer:** Disa modellerade trekropparsproblemet utan jorden genom att sätta $M_S = 1$, $R_J = 5{,}2$ och en massiv planet $M_J = 0{,}08$ med vinkelhastigheten $\omega_J = 5{,}2^{-1,5}$ för att få en tydlig och stabil gravitationell avböjning. För att förhindra orimliga energispikar vid närpassagen lades en dämpning $\varepsilon = 10^{-4}$ till i avståndsberäkningen, och integreringen utfördes med semi-implicit Euler för att undvika energidrift. Med starthastigheten $v_0 = 1{,}34$ (vilket ger

bunden startenergi $E_0 \approx -0{,}1022 < 0$) och startfasen $\phi_0 = 1{,}78$ passerade sonden strax bakom Jupiter och slungades ut i en hyperbolisk flyktbana med $E_{\text{post}} \approx +0{,}1431 > 0$. Anton tog inte med jorden. Jorden väger bara ungefär 1/300 av Jupiter och påverkar skeppet bara precis i början, medan det är passagen förbi Jupiter som avgör om skeppet kommer ut. Anton började med Hohmann-Δv och letade sedan upp rätt startvinkel för Jupiter. Med för stort h gav vissa vinklar helt orimliga energier (till exempel $E = 342$). Det var inte på riktigt, utan berodde på att skeppet kom för nära Jupiter för att stegen skulle hinna med. Fredrik gjorde sitt program enkelt att bygga olika testfunktioner runt även om det inte var nödvändigt då programmet gör det lätt att flytta runt planeterna som man vill.

2. **Hur man vet att skeppet lämnar solsystemet:** Disa följde skeppets energi över tid och såg att den efter passagen ökade från negativ ($E_0 \approx -0{,}10$) till stabilt positiv ($E \approx +0{,}14$). Eftersom energin är positiv övergår banan till en öppen hyperbel med en fart på ca 0,54 AU/tidsenhet långt bort från solen, vilket garanterar att skeppet lämnar solsystemet. Anton studerade också energikriteriet ($E \geq 0$ för flykt) och erhöll $E \approx +0{,}0055$. Att $h = 10^{-4}$ och 10^{-5} gav identiska svar bekräftar konvergensen, medan $h = 10^{-3}$ gav ett fel på över en tiopotens. Fredrik beräknade E utifrån systemets tillstånd efter plotten gjorts.